AgentSecBench: A Benchmark Framework for Evaluating Security Tools in the Age of Agentic Development

Akash Mahajan[1] Kashi KS[2] Thiyagarajan M[2] Subho Halder[3]

[1]*Appsecco* [2]*Kalmantic Labs* [3]*MatterSec*

Draft v0.2, February 2026

Abstract

The emergence of AI coding agents has collapsed the latency window that traditional application security depended on. Static analysis, dependency scanning, and audit-layer tools were architected for human-speed development –they assume code exists before inspection. Agents generate, execute, and deploy before scanners run. Meanwhile, a new generation of AI-powered security tools –exemplified by Anthropic's Claude Code Security –promises reasoning-based vulnerability detection that surpasses pattern-matching approaches. Yet no standardized benchmark exists to evaluate these tools against each other or against the new attack surfaces that agentic development introduces.

We propose **AgentSecBench**, an open benchmark framework that evaluates security tools for *coding and web application agents* across four trust dimensions: *Secure* (vulnerability detection), *Competent* (finding quality and actionability), *Accountable* (explainability and auditability), and *Surface* (coverage of novel agentic attack surfaces including MCP protocols, skill registries, and client-side exposure). We introduce a vulnerability taxonomy (AG01–AG06) mapped to MITRE ATLAS [6] techniques and OWASP Top 10 for Agentic Applications [5], with compliance alignment to NIST AI RMF, PCI DSS v4.0, and HIPAA. We distinguish between *evangelism-layer* taxonomies and *operational verification standards*, positioning AgentSecBench as both: the taxonomy defines what to measure, while per-category verification checklists define how to pass.

1 Introduction

Application security tooling is undergoing a phase transition. For two decades, the security industry operated on a stable assumption: developers write code, commit it, and scanners inspect it before or shortly after deployment. SAST tools match code against known vulnerability patterns. SCA tools check dependency manifests against CVE databases. DAST tools probe running applications against known attack signatures. The entire audit layer depends on a temporal gap between code creation and code execution.

AI coding agents have eliminated that gap.

Tools like Claude Code, Cursor, and Windsurf generate, test, and deploy code in seconds. The hallucinated npm package is installed before the scanner's next scheduled run. The misconfigured API route is live before the PR reviewer wakes up. Prevention –the foundational promise of DevSecOps –requires a window of time that no longer exists.

Simultaneously, AI is being applied to security itself. Anthropic's Claude Code Security, launched in February 2026, uses Claude Opus 4.6 to reason about codebases "the way a human security researcher would," claiming over 500 vulnerabilities found in production open-source projects that had evaded decades of expert review [1]. This represents a fundamentally different approach: not pattern matching, but semantic understanding of code behavior.

Yet the security landscape is not monolithic. White-box source analysis cannot detect what is already exposed through deployed client-side bundles. Black-box reconnaissance tools –which render SPAs, fetch JavaScript bundles, and extract secrets and endpoints – address a complementary attack surface that source-level tools are structurally blind to.

No benchmark exists to evaluate any of this. We have SWE-bench for coding capability [2]. We have nothing equivalent for security tooling in the agentic era.

1.1 Scope: Agent-Type-Specific Benchmarks

A universal benchmark for "all agents" would abstract away the specific guardrails, surfaces, and compliance requirements that differ across agent types. A coding agent's security profile is fundamentally different from a browser agent's, which differs from an agent that touches healthcare data or controls physical systems.

AgentSecBench is deliberately scoped to **coding agents and web application agents** –the agent types

that generate, deploy, and interact with web application code. The vulnerability taxonomy, test applications, and scoring methodology are designed for this domain. Agents operating in healthcare (HIPAA), autonomous vehicles, or financial trading require separate benchmarks with domain-specific guardrails, surfaces, and compliance mappings.

This specificity is a feature, not a limitation. As OWASP Top 10 demonstrated, broad evangelism documents achieve wide adoption but provide minimal actionable detail. The OWASP Application Security Verification Standard (ASVS) [9] and the emerging AI Security Verification Standard (AISVS) [8] provide operational depth by being specific. AgentSecBench follows the ASVS model: scoped, testable, and actionable for its target domain.

2 Background and Motivation

2.1 The Collapse of the Audit Layer

Traditional application security follows a linear pipeline:

```
Write → Commit → Scan → Review → Deploy
```

Every commercial security tool inserts itself somewhere in this pipeline. SAST operates between Commit and Review. DAST operates after Deploy. SCA operates at Commit time. The entire model assumes sequential, human-paced development.

Agentic development compresses this to:

```
Prompt → Generate → Execute
```

There is no commit phase. There is no review window. The agent generates code and runs it. By the time a scanner processes the output, the code is already executing –potentially in production. The inspection model itself is obsolete.

2.2 The Rise of AI-Powered Security

Claude Code Security represents the first major attempt by a foundation model provider to apply frontier reasoning capabilities to vulnerability detection. It reads code and reasons about data flows, component interactions, and logical invariants –the way an experienced security researcher would. However, this capability operates strictly in the white-box domain [1].

2.3 The Black-Box Reconnaissance Gap

Modern SPAs ship substantial application logic to the client: API endpoints, auth flow logic, hardcoded secrets, client-side access control, library versions with known CVEs, and source maps exposing original source. None of this is visible to a source-code scanner. Black-box SPA analysis tools operate in this gap, answering: "what has this application already exposed to every visitor?"

2.4 The Benchmark Gap

The security tooling landscape contains at least four categories: traditional SAST/SCA, traditional DAST, AI-powered white-box, and AI-powered black-box. No benchmark evaluates these categories against each other across the attack surfaces that matter in 2026.

2.5 The Standards Landscape

Enterprise adoption of agent security tools will be governed by existing and emerging standards. MITRE ATLAS [6] provides the threat model (15 tactics, 66 techniques, 26 mitigations), with 14 agentic-specific techniques added via Zenity Labs in October 2025 and additional agent techniques in early 2026. NIST AI RMF [7] provides the risk management framework. The OWASP Top 10 for Agentic Applications [5] provides the evangelism layer. The OWASP AISVS [8] is developing the operational verification standard.

A benchmark must map to these standards, not ignore them. AgentSecBench maps its taxonomy to ATLAS techniques, its dimensions to NIST AI RMF functions, and its compliance requirements to PCI DSS and HIPAA safeguards.

3 The AgentSecBench Framework

We propose a benchmark organized around four trust dimensions, inspired by the emerging trust infrastructure framework for agentic systems [3].

3.1 Design Principles

P1: Ground truth, not opinion. Every test case has a documented vulnerability with known type, location, severity, and correct fix.

P2: Modern stack, real patterns. Test applications use current frameworks (Next.js 15, Nuxt 4, SvelteKit) with realistic code patterns.

P3: Multi-perspective evaluation. The same vulnerability may be detectable from source or from the deployed bundle. The benchmark captures which perspective each tool operates from.

P4: Novel surfaces included. Agentic-specific vulnerability classes: MCP protocol security, skill registry poisoning, and AI-generated code artifacts.

P5: Open and extensible. Dataset, runners, and scoring

are open-source.

P6: Agent-type scoped. Deliberately scoped to coding and web application agents, not universal.

P7: Standards-mapped. Every vulnerability category maps to MITRE ATLAS techniques and OWASP Agentic Top 10.

3.2 Dimension 1: Detection (SecureBench)

Core question: Does the tool find real vulnerabilities?

The test corpus consists of 60 intentionally vulnerable web applications in three tiers:

Table 1: Test corpus composition

Tier	Count	Examples
Standard	20	SQLi in Next.js API route, XSS in React component
Complex	20	IDOR via client-side route guard, race condition in payment
Agentic	20	Hallucinated package, malicious MCP tool, prompt injection in comments

Metrics:

Table 2: Detection dimension metrics

Metric	Formula	Weight
True Positive Rate	found / total vulns	0.30
False Positive Rate	$1 -$ (FP / total findings)	0.20
Severity Accuracy	correct sev / total found	0.15
Time to Detection	norm(1 / avg seconds)	0.15
Patch Quality	correct fixes / total fixes	0.20

3.3 Dimension 2: Quality (CompetenceBench)

Core question: Are the findings actionable and correct?

Four tests: (1) Precision under load against 10 large OSS projects. (2) Adversarial robustness –prompt injection in code comments. (3) Explanation quality rated 1–5 by security engineers (Krippendorff's α). (4) Fix correctness –patches must compile, pass tests, and eliminate the vulnerability.

3.4 Dimension 3: Auditability (AccountabilityBench)

Core question: Can we trace and reproduce what the tool did?

Six criteria scored 0–5: reasoning chain, evidence linking, confidence calibration, output standardization (SARIF), reproducibility, and audit trail.

3.5 Dimension 4: Surface Coverage (SurfaceBench)

Core question: Does the tool cover the attack surfaces that matter in 2026?

Table 3: Attack surface coverage matrix

#	Surface	Trad.	AI-WB	AI-BB
S1	REST/GraphQL endpoints	$\sim$		
S2	Client-side secrets	$\times$	$\sim$	
S3	Source map exposure	$\times$	$\times$	
S4	MCP protocol	$\times$	$\sim$	$\times$
S5	Skill/plugin registries	$\times$	$\sim$	$\times$
S6	Supply chain (agentic)	$\sim$		$\times$

3.6 Composite Scoring

$$\text{AgentSec Score} = 0.35D + 0.25Q + 0.15A + 0.25S \quad (1)$$

where D = Detection, Q = Quality, A = Auditability, S = Surface Coverage. Each normalized to 0–100. Tools tagged with operating perspective: `WB`, `BB`, or `HY`.

4 Vulnerability Taxonomy and Standards Mapping

4.1 Evangelism vs. Verification

The OWASP Top 10 is an evangelism document –it raises awareness of vulnerability categories. The OWASP ASVS is an operational verification standard –it provides testable requirements at three assurance levels that organizations can certify against. PCI DSS Requirement 6.4.1 explicitly references the OWASP Top 10, making it one of the few regulatory standards that names OWASP directly [10].

Our AG01–AG06 taxonomy serves as the *evangelism layer* –communicating what new vulnerability classes exist in the agentic era. Section 4.5 introduces per-category *verification requirements* that serve as the operational layer, following the ASVS/AISVS model.

4.2 AG01–AG06 Mapped to MITRE ATLAS

Each AgentSecBench category maps to specific MITRE ATLAS techniques [6], enabling organizations already using ATLAS for threat modeling to integrate AgentSecBench findings into their existing workflows.

Table 4: AG taxonomy mapped to MITRE ATLAS techniques

ID	Category	ATLAS Techniques
AG01	Client-Side Secret Exposure	AML.T0055 (Unsecured Credentials), AML.T0083 (Creds from Agent Config)
AG02	Source Map Leakage	AML.T0000 (Search Public Materials), CWE-312
AG03	MCP Tool Injection	AML.T0053 (Plugin Compromise), AML.T0099 (Agent Tool Data Poisoning), AML.T0051.001 (Indirect Prompt Injection)
AG04	Skill Registry Poisoning	AML.T0053, AML.T0099, AML.T0058 (Publish Poisoned Models)
AG05	Hallucinated Dependency	AML.T0060 (Publish Hallucinated Entities), AML.T0062 (Discover Hallucinations)
AG06	Agent Prompt Manipulation	AML.T0051 (Prompt Injection), AML.T0054 (Jailbreak), AML.T0080 (Context Poisoning)

4.3 AG01–AG06 Mapped to OWASP Agentic Top 10

The OWASP Top 10 for Agentic Applications (December 2025) [5] defines ten risk categories for autonomous AI agents. Our taxonomy maps directly:

Table 5: AG taxonomy mapped to OWASP Agentic Top 10

AG ID	Category	OWASP Agentic
AG01	Client-Side Secrets	ASI03 (Identity & Privilege Abuse)
AG02	Source Map Leakage	ASI03 (Identity & Privilege Abuse)
AG03	MCP Tool Injection	ASI02 (Tool Misuse), ASI04 (Supply Chain)
AG04	Skill Poisoning	ASI04 (Agentic Supply Chain)
AG05	Hallucinated Dep.	ASI04 (Supply Chain), ASI05 (Code Exec.)
AG06	Prompt Manipulation	ASI01 (Goal Hijacking), ASI06 (Memory Poisoning)

4.4 ATLAS Mitigations Alignment

MITRE ATLAS defines 26 mitigations. The following are directly relevant to AgentSecBench categories:

Table 6: Relevant ATLAS mitigations

Mitigation ID	Name	AG Categories
AML.M0012	Encrypt Sensitive Info	AG01
AML.M0016	Vulnerability Scanning	AG01, AG02
AML.M0030	Restrict Tool Invocation	AG03, AG06
AML.M0013	Code Signing	AG04
AML.M0014	Verify ML Artifacts	AG04, AG05
AML.M0023	AI Bill of Materials	AG04, AG05
AML.M0020	GenAI Guardrails	AG03, AG06
AML.M0015	Adversarial Input Det.	AG06
AML.M0026	Privileged Agent Perms	AG03
AML.M0029	Human In-the-Loop	AG06

4.5 Verification Requirements

Following the ASVS model, each AG category includes testable verification requirements at two levels:

Level 1 (L1), Automated: Requirements that can be verified by automated tooling (the benchmark itself).

Level 2 (L2), Review: Requirements that require human review or organizational process verification.

Table 7: Sample verification requirements (AG01)

Level	Req ID	Requirement
L1	AG01.1	Tool detects API keys in JS bundles with $\geq$80% TPR
L1	AG01.2	Tool detects cloud credentials (AWS, GCP, Azure) with $\geq$90% TPR
L1	AG01.3	Tool reports severity as critical for production credentials
L2	AG01.4	Findings include remediation guidance (server-side env vars)
L2	AG01.5	Tool distinguishes publishable keys from secret keys

~~Full~~ verification requirements for AG01–AG06 are published in the benchmark repository.

5 Compliance Mapping

Enterprises adopting agent security tools must demonstrate compliance with existing frameworks. AgentSecBench dimensions map to the major standards enterprises will reference.

5.1 NIST AI Risk Management Framework

The NIST AI RMF [7] defines four functions: Govern, Map, Measure, and Manage. AgentSecBench dimensions align as follows:

Table 8: AgentSecBench dimensions mapped to NIST AI RMF

Dimension	NIST AI RMF	Key Subcategories
Detection	MEASURE 2.7	Security/resilience eval
Quality	MEASURE 2.5, 2.9	Validity, explainability
Auditability	GOVERN 1.5, MANAGE 4.1	Monitoring, incident resp.
Surface	MAP 4.1, MANAGE 3.1	Third-party risk, monitoring

5.2 PCI DSS v4.0 and HIPAA

For agents handling payment card data, PCI DSS v4.0 Requirement 6.4.1 explicitly references the OWASP Top 10 as an acceptable security review methodology [10]. AgentSecBench extends this: organizations can reference the AG taxonomy as supplementary to the OWASP Top 10 for agentic-specific risks.

For agents touching healthcare data, HIPAA Technical Safeguards (§164.312) require access control, audit controls, integrity verification, and transmission security. AgentSecBench's Auditability dimension directly supports HIPAA §164.312(b) (audit controls), while the Surface dimension's client-side secret detection (AG01) supports §164.312(a)(1) (encryption/access control for ePHI).

Table 9: Cross-standard compliance mapping

Domain	AISVS	NIST	PCI	HIPAA
Access Ctrl	C5	GOV 2.1	Req 7,8	§312(a),(d)
Supply Chain	C6	MAP 4.1	Req 6.2.4	n/a
Input Valid.	C2	MEA 2.7	Req 6.2.4	§312(c)
Monitoring	C12	MAN 4.1	Req 10	§312(b)
Encryption	C11	MEA 2.10	Req 3,4	§312(e)
Agent Action	C9	MAN 2.4	n/a	n/a

Note: HIPAA does not reference OWASP or NIST AI RMF directly. HITRUST CSF serves as the practical compliance bridge. Mappings are inferred, not mandated.

6 Benchmark Dataset Construction

Test applications are constructed via three methods: (1) manual seeding by security engineers, (2) CVE reproduction from real-world applications, and (3) AI-assisted generation with human-seeded vulnerabilities for the "agentic" tier.

6.1 Framework Coverage

Table 10: Target SPA frameworks

Framework	Apps	Rationale
Next.js 15	20	Dominant, server components
Nuxt 4	10	Vue equivalent, different SSR
SvelteKit	10	Growing, different compilation
React SPA (Vite)	10	Pure client-side, max exposure
Angular 19	10	Enterprise, different modules

6.2 From Vulnerable Agent to Benchmark Test Case

A common question for practitioners is: "I have a vulnerable agent. How do I turn it into a benchmark test case?" The process is:

1. **Identify the vulnerability class**: Map the vulnerability to AG01–AG06 or OWASP Agentic ASI01–ASI10.
2. **Isolate the vulnerable component**: Extract the minimum code that reproduces the vulnerability into a standalone application.
3. **Write the ground truth manifest**: Document type, severity, location (source + bundle + endpoint), detection perspective, ATLAS technique ID, and correct fix.
4. **Validate exploitability**: Confirm the vulnerability is exploitable via both manual testing and at least one existing tool.
5. **Submit**: Contribute the test case to the benchmark repository with peer review.

7 Runner Architecture

Each tool is wrapped in a standardized runner that: (1) accepts a test application (source directory or deployed URL), (2) invokes the tool with default configuration, (3) normalizes output to SARIF [11], and (4) records wall-clock time and resource consumption.

For black-box tools, applications are deployed to isolated containers. For white-box tools, the source directory is provided directly. Time budgets: 2 minutes (small), 5 minutes (medium), 10 minutes (large).

8 Preliminary Analysis

8.1 Architectural Capability Mapping

White-Box AI (Claude Code Security): Reasons about code semantics; detects business logic flaws; suggests fixes. Blind to build output and runtime exposure.

Black-Box AI (SPA Hacking Agent): Sees what at-

tackers see; detects client-side secrets, source maps, deployed endpoints. Cannot reason about server-side logic.

Traditional SAST: Fast, deterministic, extensive rules. No semantic understanding; blind to deployment. **Traditional DAST**: Tests running applications. Probe-based, limited JS analysis, slow.

8.2 The Complementarity Thesis

We hypothesize: (1) AI white-box dominates Detection and Auditability; (2) AI black-box dominates Surface S1–S3; (3) no tool scores above 70 across all four dimensions; (4) white-box + black-box AI combination outscores any single tool. If confirmed, the agentic era requires **layered AI security**.

9 Implications for the Security Industry

9.1 The Trust Infrastructure Thesis

The benchmark dimensions map to emerging trust problems [3]: Secure → Detection, Competent → Quality, Accountable → Auditability, Verified → Surface Coverage.

9.2 Pricing Model Disruption

Per-seat pricing shrinks with agent-driven headcount reduction. Per-scan pricing explodes with continuous generation. The benchmark enables **per-vulnerability-class coverage** pricing.

9.3 The Composition Architecture

The winning architecture is a **composition layer**: routing source to white-box analysis, deployed apps to black-box reconnaissance, build artifacts to supply-chain analysis, and agent configs to agentic surface analysis.

10 Related Work

Agent Security Bench (ASB) [12] formalizes attacks and defenses on LLM-based agents with 400+ tools, 27 attack methods, and 7 metrics across 10 scenarios. ASB evaluates whether *agents themselves are vulnerable to attacks* (prompt injection, backdoors, memory poisoning) –measuring the highest attack success rate at 84.30% while showing limited defense effectiveness. AgentSecBench evaluates whether *security tools can detect vulnerabilities* –a complementary but architecturally distinct goal. ASB's test subjects are agents; ours are security scanners.

SEC-bench [13] benchmarks LLM agents on real vulnerability patching tasks. It tests whether agents can *fix* vulnerabilities, not whether security tools can *find* them.

WASP [14] benchmarks web agent security against prompt injection specifically –narrower than our multi-surface, multi-tool evaluation.

SecureAgentBench [15] evaluates whether code agents *generate* secure code –measuring agent output quality, not detection tool capability.

MITRE ATLAS [6] provides the threat taxonomy (techniques and mitigations) but is not a benchmark –it does not score tools or provide test datasets. AgentSecBench operationalizes ATLAS by mapping techniques to testable scenarios.

OWASP AISVS [8] develops verification requirements for AI systems across 13 categories. AgentSecBench complements AISVS by providing the benchmark dataset and scoring engine that verifies whether tools meet AISVS-aligned requirements.

AgentSecBench is the first benchmark designed for cross-category comparison of security *detection tools* – both AI-powered and traditional –across the full range of agentic-era attack surfaces, with explicit standards mapping.

11 Limitations and Future Work

Limitations: The initial 60-app dataset cannot cover all vulnerability classes. Auditability scoring relies partially on human evaluation. The benchmark evaluates tools in isolation. MCP and skill registry surfaces (S4, S5) are based on threat modeling, not observed attacks at scale. AG01 and AG02 are web application concerns that predate AI –their inclusion reflects their prevalence in SPA agent attack chains, not novelty. HIPAA mappings are inferred via HITRUST CSF, not directly mandated.

Future work: (1) Quarterly dataset updates as agentic surfaces evolve. (2) An adversarial track evaluating evasion resistance. (3) Composition scoring for tool combinations. (4) Domain-specific benchmark variants for healthcare agents, financial agents, and infrastructure agents. (5) A cost-effectiveness fifth dimension for ROI comparison. (6) Alignment with the OWASP AI Security Summit at RSAC 2026 [17] for community validation and adoption.

12 Conclusion

The security industry is undergoing a structural transition. The audit layer is being compressed out of existence by AI coding agents. New AI-powered tools offer capabilities traditional pattern-matching cannot match, but they operate from fundamentally different perspectives

with complementary blindspots.

The industry needs a benchmark –scoped to specific agent types, mapped to existing standards, and actionable at both the evangelism and verification layers. Not a universal abstraction that becomes a risk framework, but a domain-specific instrument that tells a CISO whether a coding agent's security tooling meets measurable requirements.

AgentSecBench is that instrument. By evaluating tools across detection, quality, auditability, and surface coverage –mapped to MITRE ATLAS, OWASP Agentic Top 10, NIST AI RMF, and enterprise compliance frameworks –it provides the empirical foundation for security architecture decisions in the agentic era.

The audit layer is dying. The trust layer is emerging. The benchmark tells you who to trust –and what to verify.

References

[1] Anthropic. "Making frontier cybersecurity capabilities available to defenders." February 2026. https://www.anthropic.com/news/claude-code-security

[2] C. E. Jimenez et al. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" *ICLR*, 2024.

[3] P. Gosavi. "SaaS Cybersecurity is Dead." 2026. As analyzed in "Follow the White Trust" (unpublished).

[4] OWASP Foundation. "OWASP Top 10:2021." https://owasp.org/Top10/

[5] OWASP Foundation. "OWASP Top 10 for Agentic Applications." December 2025. https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

[6] MITRE. "MITRE ATLAS: Adversarial Threat Landscape for Artificial Intelligence Systems." https://atlas.mitre.org/

[7] NIST. "AI Risk Management Framework (AI RMF 1.0)." AI 100-1, January 2023. https://www.nist.gov/itl/ai-risk-management-framework

[8] OWASP Foundation. "AI Security Verification Standard (AISVS)." Draft, 2026. https://owasp.org/www-project-artificial-intelligence-security-verification-standard-aisvs-docs/

[9] OWASP Foundation. "Application Security Verification Standard (ASVS) v4.0." https://owasp.org/www-project-application-security-verificat

[10] PCI Security Standards Council. "PCI DSS v4.0." March 2022.

[11] OASIS. "Static Analysis Results Interchange Format (SARIF) Version 2.1.0." 2018.

[12] H. Zhang et al. "Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents." *ICLR*, 2025. *arXiv:2410.02644*.

[13] "SEC-bench: Automated Benchmarking of LLM Agents on Real-World Software Security Tasks." *arXiv:2506.11791*, 2025.

[14] "WASP: Benchmarking Web Agent Security Against Prompt Injection Attacks." *arXiv:2504.18575*, 2025.

[15] "SecureAgentBench: Benchmarking Secure Code Generation under Realistic Vulnerability Scenarios." *OpenReview*, 2025.

[16] NIST. "National Vulnerability Database." https://nvd.nist.gov/

[17] OWASP GenAI Project. "RSAC 2026 OWASP AI Security Summit." https://genai.owasp.org/event/rsac-conference-2026-owasp-ai-security-sum

[18] Zenity Labs. "Zenity Labs and MITRE ATLAS Collaborate to Advance AI Agent Security." October 2025.